FFmpeg has very powerful scale filter which can accomplish various tasks. Below lists some examples.
More info in the official doc:
https://ffmpeg.org/ffmpeg-filters.html#scale
https://ffmpeg.org/ffmpeg-scaler.html

["swscale" related:
https://github.com/FFmpeg/FFmpeg/blob/master/libswscale/options.c
https://github.com/search?type=code&q=repo:FFmpeg/FFmpeg+%22Lanczos%22
https://github.com/FFmpeg/FFmpeg/raw/master/doc/swscale.txt ("initFilter")
https://github.com/FFmpeg/FFmpeg/blob/4ec45aca36ff6c7c7767536ce7f4a855f8b37fb0/libswscale/utils.c#L405-L415
https://github.com/FFmpeg/FFmpeg/blob/3e064f52eb368a373ded6e3704fcf29f1db3ff12/libswscale/swscale.h#L207-L212
https://github.com/FFmpeg/FFmpeg/blob/66592e8b100f4b521b48962f15940e19ef6eed48/libswscale/output.c#L175

For reference:
https://github.com/FFmpeg/FFmpeg/commit/2f2f73dc3ae3fadbf040eb73f217213b1c99ff39
https://github.com/FFmpeg/FFmpeg/blob/6d75d44d905ec8047346c6f74c48d81549f31891/libavfilter/vf_scale_npp.c#L1002-L1010
https://github.com/FFmpeg/FFmpeg/blob/6d75d44d905ec8047346c6f74c48d81549f31891/libavfilter/vf_zscale.c#L998-L1004]

Warning:

- https://trac.ffmpeg.org/ticket/10993
- When using "-lavfi" ("-filter_complex"):
  The default scaling flags wouldn't be applied, so the default scaling algorithm would be "bilinear" instead of "bicubic". [Changed to "bicubic" since 9f14396a5103ec80893db801035ece5d14c0d3c5.]
  To achieve the same: specify the algorithm via "flags=bicubic" alike.
- In some cases, FFmpeg may set the Sample Aspect Ratio to compensate for ratio change.
  Expressly state SAR 1:1 to make things work intended.
  E.g.

  ```
  ffmpeg -i "${In}" -vf "scale=320:240,setsar=1:1" "${Out}"
  ```

  Note above highly misleading, for "setsar=1:1" actually means "setsar=r=1:max=1":
  Which happened to achieve the desired SAR 1:1.
  For "setsar=2:3" alike it shall give the wrong SAR 2:1, or else.
  Correct syntax for SAR 2:3 is "setsar=2/3"; or "setsar=1" for 1:1.

- When going from BGR (not RGB) to yuv420p the conversion may be off-by-1. Use "-sws_flags +accurate_rnd" alike or "zscale" to avoid.
  [^ With "swscale", for many colors. See #979 for further caveats.]
- yuvj***p pixel formats are deprecated. Yet for x265 the workaround was implemented, but not for jpeg2000 and AV1. For those -vf scale=out_range=pc should be used.
- Conversion from YUV limited to RGB 16bpc (bit-per-component) is broken: use "zscale" instead of "swscale". [Via "-vf" alike.]
- Limited range RGB is outright unsupported.
- Dither can be forced off using "-sws_dither none". (or temporary workaround `-lavfi "scale=sws_dither=none"` alike)
- Remember YUV 4:4:4 8bpc wouldn't suffice to preserve RGB 8bpc: 10bpc is required.
- The default matrix for untagged input/output would be limited BT.601. [See also: <colorspace>]

All examples below, the source ("input.jpg") shall be:
(~ 53.82 KiB; JPEG: YUV 4:2:0, 535x346)

t a g s
scale

Simple Rescaling

If you need to simply resize your video to a specific size (e.g 320x240), you can use the scale filter in its most basic form:

```
ffmpeg -i input.avi -vf scale=320:240 output.avi
```

Same works for images too:

```
ffmpeg -i input.jpg -vf scale=320:240 output_320x240.png
```

The resulting image will look like this:

As you can see, the aspect ratio is not the same as in the original image, so the image appears stretched.

Keeping the Aspect Ratio

If we'd like to keep the aspect ratio, we need to specify only one component, either width or height, and set the other component to -1. For example, this command line:

```
ffmpeg -i input.jpg -vf scale=320:-1 output_320.png
```

will set the width of the output image to 320 pixels and will calculate the height of the output image according to the aspect ratio of the input image.

The resulting image will have a dimension of 320x207 pixels.

Some codecs require the size of width and height to be a multiple of *n*. You can achieve this by setting the width or height to *-n*:

```
ffmpeg -i input.jpg -vf scale=320:-2 output_320.png
```

The output will now be 320x206 pixels.

Using Variables

There are also some useful variables which can be used instead of numbers, to specify width and height of the output image.

For example, if you want to stretch the image in such a way to only double the width of the input image, you can use something like this (`iw` = input width, `ih` = input height):

```
ffmpeg -i input.jpg -vf "scale=iw*2:ih" input_double_width.png
```

The output image will look like this:

If you want to half the size of the picture, just multiply by .5 or divide by 2:

```
ffmpeg -i input.jpg -vf "scale=iw*.5:ih*.5" input_half_size.png
ffmpeg -i input.jpg -vf "scale=iw/2:ih/2" input_half_size.png
```

Avoiding Upscaling

Sometimes you want to scale an image, but avoid upscaling it if its dimensions are too low. This can be done using `min` expressions:

```
ffmpeg -i input.jpg -vf "scale='min(320,iw)':'min(240,ih)'" input_not_upscaled.png
```

The output width will be evaluated to be the minimum of 320 and the input width. If you have an imput image that is only 240 pixels wide, the result of the `min` function will be 240 - this will be your target value.

Fitting into a Rectangle / Statically-sized Player

Sometimes you need to scale the input image so that it fits into a specified rectangle, e.g. when consolidating material from different sources.

Achievable with "force_original_aspect_ratio". Of 3 possible values:
|0| "disable" (default)
|1| "decrease": auto-decrease output dimensions on need.
|2| "increase": auto-increase output dimensions on need.

This allows you to force the image to fit into a 320x240 box, downscaling it with the correct aspect ratio:

```
ffmpeg -i "input.jpg" -vf "scale=w=320:h=240:force_original_aspect_ratio=decrease" "output_320.png"
```

This produces our 320x207 image that we had seen before.

You may have additional constraints such as adding black bars (pillar- and letterboxing) to fill up the remaining space when scaling to a certain rectangle.

You can additionally add a black border using the "pad" filter:

`ffmpeg -i "input.jpg" -vf "scale=320:240:force_original_aspect_ratio=1,pad=320:240:(( (ow - iw)/2 )):(( (oh - ih)/2 ))" "output_320_padding.png"`

t a g s
scale

More examples:
https://superuser.com/questions/547296#1136305

scale2ref

If you need to scale one video to match another without pre-defining the dimensions, see the scale2ref filter.

Two videos are provided for input and output. The 1st input is scaled and the 2nd input is unchanged, but used for reference. For example, to put 2 videos side-by-side and have the 2nd video scale to match the first:

```
ffmpeg -i vid1.mp4 -i vid2.mp4 -filter_complex "[1:v][0:v]scale2ref=oh*mdar:h=in_h:[v1][v0];[v0][v1]hstack[vo]"
```

Specifying scaling algorithm

You can specify which algorithm is used for resizing with the `-sws_flags` option. For example, to use bilinear instead of the default bicubic scaling:

```
ffmpeg -i test.tif -vf scale=504:376 -sws_flags bilinear out.bmp
```

List of scaling algorithms with their defaults, if any:
fast_bilinear
bilinear
bicubic
experimental
neighbor
area
bicublin, Bicubic scaling algorithm for the luma component, bilinear for chroma components.
gauss
sinc
lanczos, The default width (alpha) is 3 and can be changed by setting param0.
spline

> Explanation on scaling algorithms: (vague incomplete)
> https://ffmpeg.org/ffmpeg-filters.html#Scaling
> https://libplacebo.org/options/#scaling

See the lead section documentation for more info.

Setting param0 for lanczos is done via

```
-vf scale=1920x1080:flags=lanczos:param0=3
```

To set multiple flags you can combine them with a `+` sign. For example:

```
-sws_flags lanczos+accurate_rnd+full_chroma_int
```

You can also specify the options directly in the `scale` filter, for example:

```
-vf scale=1920x1080:flags=lanczos
```

Chroma sample location

FFmpeg does not see the chroma sample location on either limited or full range file, so you should specify it by hand when going to .rgb (or when upsampling from 4:2:0 to 4:4:4 in .yuv):

```
ffmpeg -i cosmos.mkv -vf 'scale=in_h_chr_pos=0:in_v_chr_pos=128' -pix_fmt rgb24 qnfvarejwnfe3.rgb //for left
ffmpeg -i cosmos.mkv -vf 'scale=in_h_chr_pos=0:in_v_chr_pos=0' -pix_fmt rgb24 qnfvarejwnfe3.rgb //for top-left
```

Recently introduced in_chroma_loc and out_chroma_loc can be used as well.

```
ffmpeg -i cosmos-p5.mkv -vf scale=in_chroma_loc=left -pix_fmt rgb24 newway.rgb
```

```
ffmpeg -i cosmos-p5.mkv -vf scale=in_chroma_loc=topleft -pix_fmt rgb24 newway.rgb
```

To create BT.2020-NCL matrix with top-left chroma this can be used (if the source image is not using bt.2020 primaries the color management must be done first on linear data, also correct transfer must be tagged, either SDR or HDR one, again this requires color management):

```
-vf scale=out_color_matrix=bt2020nc:out_h_chr_pos=0:out_v_chr_pos=0
```

To change chroma sample location in VUI in HEVC to 0 (left) with -c copy this can be used (2 would be top-left, -1 will remove any sample location from VUI, all 0, 1, 2, 3, 4, 5 sample locations are possible):

```
ffmpeg -i file.mp4 -c copy -bsf:v hevc_metadata=chroma_sample_loc_type=0 file1.mp4
```

Also, #9693 says that if you want to write jpeg file you need to use zscale and specify matrix in and out and chroma location in and out -vf zscale=min=709:m=170m:cin=left:c=center

▶ **Attachments** (5)

Last modified on Sep 10, 2024, 3:49:35 AM

t a g s

scale